

Session 7 — Contribuer efficacement : bonnes pratiques

Objectifs de la séance

- Rédiger une pull request de qualité.
- Écrire des messages de commit clairs et normalisés.
- Communiquer efficacement avec les mainteneurs.
- Participer à la *code review* (recevoir et donner).
- Lancer son propre projet open source.
- Gérer les versions et les *releases*.

Partie 1 — L'anatomie d'une bonne PR

Qu'est-ce qu'une bonne PR ?

Caractéristiques d'une PR qui sera mergée rapidement :

Caractéristiques à viser : un seul changement logique ; taille raisonnable (< 500 lignes) ; bien documentée ; testée ; conforme aux standards du projet.

Objectifs sous-jacents : faciliter la review, minimiser les risques de régression, accélérer le merge, respecter le temps des reviewers.

Structure d'une PR

Titre : court, descriptif, au présent.

-  « *Fix memory leak in image parser* »
-  « *Fixed stuff* »

Description — modèle simple :

What does this PR do?

[Description claire du changement]

Why is this change needed?

[Contexte, référence à l'issue]

How to test?

[Instructions pour tester]

Screenshots (if applicable)

[Captures d'écran]

Bonnes pratiques pour les PRs

1. **Une PR = un changement** — pas de PR fourre-tout.
2. **Référencer l'issue** — Fixes #123 ou Closes #456 pour la fermer automatiquement.
3. **Petites PRs > grosses PRs** — plus faciles à reviewer.
4. **Expliquer le pourquoi** — le code montre le *quoi*, la description montre le *pourquoi*.
5. **Auto-review avant de soumettre** — relire son propre diff.
6. **Tests inclus** — prouver que ça marche.

Ce qui ralentit une PR

Problèmes courants	Solutions
PR trop grosse	Découper en plusieurs PRs
Pas de description	Utiliser le gabarit du projet

Problèmes courants	Solutions
Changements non liés mélangés	Une PR par sujet
Tests manquants	Ajouter les tests
Conflits avec main	Rebaser régulièrement

Partie 2 — Messages de commit

Pourquoi c'est important

- **Historique lisible** pour comprendre l'évolution du projet.
- **Debugging** : `git bisect`, `git blame`.
- **Changelog automatique** : génération des *release notes* à partir des messages.
- **Communication** avec l'équipe — et avec votre futur vous.

Structure canonique

```
<type>(<scope>): <subject>
<ligne vide>
<body>
<ligne vide>
<footer>
```

Exemple :

```
fix(parser): handle empty input gracefully
```

Previously, passing an empty string would cause a crash.
Now it returns an empty result.

Fixes #42

Conventional Commits

Standard de plus en plus adopté — simplifie les *changelogs* automatiques et le *semver* (cf. partie 7).

Type	Usage
feat	Nouvelle fonctionnalité
fix	Correction de bug
docs	Documentation uniquement
style	Formatage, pas de changement de code
refactor	Refactoring sans changement de comportement
test	Ajout / modification de tests
chore	Maintenance, dépendances, CI

Spécification : <https://www.conventionalcommits.org>.

Règles d'or

1. **Première ligne < 50 caractères.**
2. **Impératif** : « *Add* », pas « *Added* » ou « *Adds* ».
3. **Pas de point à la fin** du sujet.
4. **Corps à 72 caractères** par ligne.
5. **Expliquer le pourquoi** dans le corps.
6. **Référencer les issues** dans le *footer*.

Exemples

Mauvais	Bon
fix bug	fix(auth): prevent session timeout during file upload
WIP	feat(api): add rate limiting to public endpoints
changes	docs: update installation instructions for Windows

Partie 3 — Communication

La communication asynchrone

L'open source fonctionne en **asynchrone** : fuseaux horaires, bénévoles, temps de réponse variables. Implications : être clair et complet dès le premier message, ne pas attendre de réponse immédiate, éviter les *ping* trop fréquents.

Règles

À faire : être poli et respectueux ; fournir le contexte nécessaire dès le premier message ; poser des questions claires ; accepter le feedback ; remercier les reviewers.

À éviter : ton agressif ou impatient ; messages du type « ça marche pas » sans détails ; relancer trop vite ; prendre les critiques personnellement ; argumenter indéfiniment.

Anti-patterns

- **Don't ask to ask** — « *Quelqu'un peut m'aider ?* » → poser directement la question avec le contexte.
- **XY Problem** — demander comment faire X alors qu'on veut Y → expliquer l'objectif réel.
- **Help vampire** — demander de l'aide sans effort préalable → montrer ce qu'on a déjà cherché.

Comment poser une bonne question

Ce que j'essaie de faire
[objectif]

Ce que j'ai essayé
[tentatives]

Ce qui se passe
[comportement actuel, erreurs, logs]

Ce que j'attendais
[comportement souhaité]

Environnement
[versions, OS, configuration]

Partie 4 — Code review

Recevoir une review

La *code review* **n'est pas une attaque personnelle**. Mindset : le reviewer essaie d'améliorer le projet, vous apprenez à chaque review, les mainteneurs ont une vision globale, un refus n'est pas un échec.

Types de commentaires

Type	Signification	Réaction
Suggestion	« <i>Consider doing X</i> »	Évaluer, appliquer si pertinent

Type	Signification	Réaction
Question	« <i>Why did you...?</i> »	Expliquer votre raisonnement
Nitpick (nit:)	Détail mineur	Corriger sans discuter
Bloquant	« <i>This must be changed</i> »	Obligatoire avant merge
Praise	« <i>Nice approach!</i> »	Dire merci

Répondre aux commentaires

Pour chaque commentaire : lire attentivement, comprendre le point soulevé, **répondre OU corriger OU discuter**.

Exemples de réponses :

- « *Good point, fixed in abc123* »
- « *I chose this approach because... What do you think?* »
- « *I see your point, but X because Y. Happy to change if you prefer.* »

La review comme contribution

La *code review* **n'est pas réservée aux mainteneurs**. Vous pouvez contribuer en reviewant les PRs des autres, même sans droits de commit.

Dans le projet Subversion, Greg Stein s'est mis à reviewer **chaque commit** qui entrait dans le dépôt, sans jamais demander aux autres de faire pareil. Rapidement, d'autres contributeurs ont suivi. — Karl Fogel, *Producing Open Source Software*.

Avant de reviewer : vérifier que le projet **accueille** les reviews externes, comprendre les attentes.

Donner une review

Soyez constructif : expliquer le pourquoi de chaque commentaire, proposer des alternatives concrètes, reconnaître le bon travail, distinguer clairement bloquant et simple suggestion.

Évitez : commentaires vagues, ton condescendant, *bikeshedding* (détails infinis sur des sujets mineurs), bloquer une PR sans explication.

Partie 5 — Pièges à éviter

Erreurs fréquentes

1. **Commencer à coder sans discuter** — pour les grosses features, discuter d'abord dans l'issue.
2. **PR massive** — impossible à reviewer, souvent abandonnée.
3. **Ignorer les guidelines** — lire CONTRIBUTING.md.
4. **Push force sans prévenir** — peut casser la review en cours.
5. **Disparaître après la PR** — répondre aux commentaires.

Avant de commencer à coder

1. **Vérifier que l'issue n'est pas prise** — quelqu'un y travaille-t-il déjà ? Y a-t-il une PR en cours ou abandonnée ?
2. **Signaler votre intention** — commenter : « *I'd like to work on this* ».
3. **Synchroniser le fork** — git fetch upstream && git rebase upstream/main avant de créer la branche.
4. **Comprendre le contexte** — lire les discussions, vérifier les PRs liées.

Garder son fork à jour

Pourquoi c'est important : éviter les conflits massifs, baser le code sur la version courante, s'assurer que les tests passent.

Moment	Obligatoire ?
Avant de créer une branche	✓
Pendant le travail	Recommandé
Avant de soumettre la PR	✓
Quand le reviewer le demande	✓

Le Signed-off-by et le DCO

Certains projets (Linux, CNCF) exigent un DCO. Un `git commit -s -m "..."` ajoute automatiquement :

Signed-off-by: Your Name <your@email.com>

Signification : « *je certifie avoir le droit de soumettre ce code.* » Détails en session 5.

Gérer un rebase

Votre PR a des conflits avec main ?

```
git fetch upstream
git rebase upstream/main
```

```
# Résoudre les conflits
git add .
git rebase --continue
```

```
git push --force-with-lease origin ma-branch
```

Préférer `--force-with-lease` à `--force` : refuse de pousser si quelqu'un d'autre a poussé entre temps.

Partie 6 — Démarrer son propre projet

Quand ouvrir son code ?

Contextes courants : application personnelle ou académique ; bibliothèque réutilisable ; outil non-cœur-de-métier d'une entreprise ; logiciel livré à un client sans besoin d'exclusivité ; produit principal d'un éditeur (modèle Red Hat).

Motivations possibles : partager (générosité) ; recevoir des contributions extérieures ; rendre le code auditable ; attirer des développeurs et recruter ; créer un standard commun avec ses pairs.

Ouvrir tôt, pas « quand ce sera prêt »

■ Le projet ne sera **jamais** suffisamment prêt.

Pourquoi ne pas attendre : plus on attend, plus le nettoyage est lourd ; on peut recevoir des contributions plus tôt qu'imaginé ; ouvrir le code **n'oblige pas** à écrire une documentation exhaustive, à être disponible en permanence, ni à accepter toutes les contributions.

Avant de recevoir des contributions : choisir une licence (session 5).

Checklist de lancement

Étape	Détail
Choisir une licence	Commencer permissif (MIT) en cas de doute — on peut toujours ajouter de la restriction plus tard
Créer le dépôt	Sur une forge (GitHub, GitLab, Codeberg...)
README.md	Mission en une phrase, résumé en un paragraphe, installation, usage
LICENSE	Fichier avec le texte complet
CONTRIBUTING.md	Comment contribuer, standards, processus
.gitignore	Adapté au langage / framework
CI/CD	Au minimum : lancer les tests automatiquement

Choix technologiques

Règle clé : réduire les besoins d'apprentissage.

Moins il faut apprendre de nouvelles choses pour contribuer → plus il y a de contributeurs potentiels, et plus vite ils sont productifs. En pratique :

- Langage, framework, outils de build : **standards** de l'écosystème.
- Éviter les solutions « maison » sans justification.
- Utiliser les outils que la communauté cible connaît déjà.

Documentation progressive

Ne pas essayer de tout documenter d'un coup. Procéder par niveaux :

1. **Mission en une phrase** — « *X is a Y that does Z* ».
2. **Résumé en un paragraphe** — problème résolu, public cible.
3. **Quick start** — installation et premier usage en 5 minutes.
4. **Exemples** — cas d'usage concrets, démos, captures.
5. **Référence** — API, configuration, options.
6. **Guide du développeur** — architecture, comment contribuer au code.

■ La documentation **utilisateur est essentielle** ; la documentation **développeur est idéale**.

Annoncer le projet

Progressivement :

1. En parler à des connaissances intéressées.
2. Répondre à des besoins exprimés sur des forums en mentionnant le projet.
3. Annonce officielle lors de la première *release*.

Canaux : mailing-lists spécialisées, Reddit, Hacker News, Lobsters, forums du langage/framework, Mastodon, conférences, meetups.

Partie 7 — Gestion des versions (*release management*)

Pourquoi produire des versions ?

Le code est déjà sur un dépôt public. Alors pourquoi des versions numérotées ?

- Désigner une version **recommandée**.
- Communiquer les **changements**.
- Distribuer via les **gestionnaires de paquets** (npm, pip, apt...).
- Produire des **installeurs** / exécutables.

- Permettre aux utilisateurs de choisir **quand mettre à jour**.

Versionnement sémantique (*semver*)

Format : **MAJEUR.MINEUR.CORRECTIF** (ex : 3.12.7).

Incrément	Quand ?	Exemple
MAJEUR	<i>Breaking change</i>	2.0.0 → 3.0.0
MINEUR	Nouvelle fonctionnalité, compatible	3.1.0 → 3.2.0
CORRECTIF	Bug fix, compatible	3.2.1 → 3.2.2

Règles importantes :

- Versions **0.x** : tout est permis (pas encore stable).
- Après 1.0 : le numéro est un **contrat** avec les utilisateurs.
- Le point n'est **pas** un séparateur décimal : après 1.9 vient 1.10.

<https://semver.org>.

Versionnement calendaire (*calver*)

Alternative, basée sur la **date** :

Projet	Format	Exemple
Ubuntu	YY.MM	24.04
pip	YY.MINOR	24.0
Chrome	MAJOR incrémental rapide	125

Pertinent quand la compatibilité arrière est moins cruciale que la fraîcheur (distributions, navigateurs). <https://calver.org>.

Release notes et changelog

Standard « Keep a Changelog » (<https://keepachangelog.com>) :

Catégorie	Description
Added	Nouvelles fonctionnalités
Changed	Modifications (impact compatibilité possible)
Deprecated	Fonctionnalités bientôt supprimées
Removed	Fonctionnalités supprimées
Fixed	Corrections de bugs
Security	Corrections de vulnérabilités

GitHub Releases

Un `git tag` + une page GitHub dédiée avec notes de version auto-générées (à partir des PRs), fichiers attachés (binaires, docs), versions préliminaires (beta, RC), notifications aux *watchers*.
Bonnes pratiques : compléter avec un `CHANGELOG.md`, créditer les contributeurs, catégoriser via les labels des PRs, automatiser via GitHub Actions.

Branches et cycles de release

Deux approches pour le rythme :

Approche	Description	Exemple
<i>Feature-based</i>	Release quand les features sont prêtes	Projets jeunes
<i>Calendar-based</i>	Cycle fixe (6 semaines, 6 mois...)	Ubuntu, Chrome, Rust

Backporting : appliquer un correctif de main sur une branche stable (1.0.x, 1.1.x...).

Ce qu'il faut retenir

1. **PRs** : petites, bien décrites, testées.
2. **Commits** : messages clairs, format Conventional Commits.
3. **Communication** : asynchrone, polie, complète.
4. **Review** : contribuer à la review, pas seulement la recevoir.
5. **Lancer un projet** : ouvrir tôt, documenter progressivement, choisir des outils standards.
6. **Releases** : semver, changelog, cycles prévisibles.

Checklist avant de soumettre une PR

- ☐ La PR ne contient qu'un changement logique.
- ☐ Les tests passent localement.
- ☐ Le code respecte les *guidelines* du projet.
- ☐ Le message de commit est clair.
- ☐ La description explique le *pourquoi*.
- ☐ L'issue liée est référencée.
- ☐ J'ai relu mon propre diff.

Pour aller plus loin

- Karl Fogel, *Producing Open Source Software* : <https://producingoss.com>.
- Conventional Commits : <https://www.conventionalcommits.org>.
- Semver : <https://semver.org>.
- Keep a Changelog : <https://keepachangelog.com>.